

Click-to-Source

Technology Stack — What, Why, and When Each Tool Is Used

Punith P | August 2026

This document explains every tool and library in the Click-to-Source stack: what it is, why it was chosen over alternatives, and — critically — at exactly which development stage it enters the project. The stack grows incrementally, with each stage adding only what that stage strictly needs. It is intentionally selected to minimize expected replacement as the project evolves — though no plan can promise a library will never need to be swapped; the goal is to choose deliberately enough that this stays the exception, not the norm.

Guiding principle: keep Stage 1 (proving the core mechanism) as close to zero-setup as possible, so a fundamental blocker can be discovered in days with almost nothing built — not after weeks of tooling investment.

Stack at a Glance

Layer	Tool / Library	Role
Language	TypeScript	Everything is written in TS for type-safe provenance records.
Core engine	three.js (Raycaster)	Click → object identification.
Target framework	React Three Fiber	The ecosystem this tool is built for; engine stays framework-agnostic underneath.
Overlay UI	React + Zustand	The in-app panel and its state.
Highlighting	three.js OutlinePass	Visual outline around the selected mesh.
File I/O	Node fs (via a small Vite plugin)	Reads and writes source files on disk.
Locate edit target	Babel (minimal parse)	Finds the exact character range of one argument inside a file.
Precise edits	magic-string	Patches that exact range without disturbing the rest of the file.
Full instrumentation	Babel (traverse, types)	AST transform that auto-tags every generator call (post-MVP).
AI integration	@modelcontextprotocol/sdk	Exposes the engine as MCP tools for AI agents.
Packaging	tsup	Bundles the package for npm publishing.
Testing	Vitest	Fast tests reusing Vite's transform pipeline.

Deep Dive: What Each Tool Does and Why

TypeScript

The language for the entire project, engine and UI alike. The core data structure this tool revolves around — a provenance record of `{ file, function, line, args }` — is exactly the kind of structured, precise data that benefits from static typing: a typo in a field name or a wrong argument shape fails at compile time instead of silently breaking resolution at runtime. It's also already the language of the R3F ecosystem this tool targets, so there's no context-switching cost for either the builder or the eventual users.

three.js — Raycaster

The mechanism behind every click. `THREE.Raycaster` is built into `three.js` itself — no extra dependency needed. A screen-space click is converted to normalized device coordinates, a ray is cast from the camera through that point, and the raycaster returns the closest intersected mesh. This is pure, deterministic geometry math — no GPU rendering is required to compute a hit, which matters on constrained hardware.

React Three Fiber

Not a dependency of the core engine itself, but the ecosystem this tool is built *for* — the target developer's app is assumed to be an R3F project. The engine logic (raycast, resolve, read/write source) is kept framework-agnostic in plain TypeScript wherever possible, with only a thin R3F-specific wrapper (e.g. a `useClickToSource()` hook) on top. This keeps the door open to supporting raw `three.js` projects later without rewriting the engine.

React + Zustand

React renders the overlay panel itself — the Generation Trace UI showing resolved file, function, and argument values. Zustand manages the overlay's own state (which object is selected, whether the panel is open). Zustand specifically because it's lightweight, has effectively zero boilerplate compared to Redux, and is built by the same team behind React Three Fiber — so it composes naturally with the rest of the target ecosystem instead of introducing an unfamiliar state pattern.

three.js OutlinePass

Provides the visual highlight around a clicked mesh. Part of `three.js`'s own `examples/postprocessing` module, so no third-party dependency is needed — just an additional import. Chosen over a plain bounding-box helper because it reads as a more polished, intentional selection indicator, closer to what professional 3D tools (Blender, Unity) use for object selection.

Node fs (via a minimal Vite plugin)

Source file reading and writing has to happen outside the browser, since a web page cannot touch the filesystem directly. A small Vite plugin runs on the developer's own local dev server and exposes two endpoints the overlay panel calls over local HTTP: one to read a file's contents, one to write an edited value back. This reuses a well-established pattern — Vite's own "open in editor" feature and various framework devtools use the same local-endpoint approach.

Babel (minimal parse) + magic-string — how an edit actually gets written

These two tools work together as a pipeline, not as alternatives — worth being precise about this, since it's easy to frame them as competing choices. To patch a value like the `0.35` inside `createTerrain({ amplitude: 0.35 })`, the exact character range of that value first has to be located inside the file — and that is inherently an AST-level task, not something `magic-string` can do on its own, since `magic-string` has no understanding of what a "parameter" or "argument" is. The real pipeline is:

Babel parses the file & locates the node → magic-string patches that exact character range → file is written

Babel's role here is deliberately narrow at this stage — just enough parsing to find one argument's location, not the full generator-tagging transform described below. That means this minimal parsing step is needed as early as Stage 3 (the MVP's basic edit-and-write-back feature), earlier than the doc previously implied. Once the node is located, magic-string does the actual writing: re-parsing the whole file into a full AST, changing one value, and re-generating the entire file text risks silently reformatting the file or dropping comments, so the surgical text-level patch is preferred for the write step specifically. This matters for developer trust — an edit tool that quietly reformats unrelated code erodes confidence fast.

Babel (full instrumentation: traverse, types)

Beyond the narrow node-locating use above, Babel's full role is powering the automatic build-time instrumentation that replaces manual tagging — the same underlying toolchain React DevTools' own JSX source-injection plugin uses to auto-attach file/line metadata to every element, with zero manual annotation from the developer. Here, a Vite/Babel plugin walks the AST at build time, identifies calls to designated generator functions, and wraps them to capture both the call site *and* the resolved runtime arguments — giving per-instance precision, not just per-line precision. This fuller transform is deliberately introduced only after the manual-tagging MVP is proven (Stage 6.5), since it's the most complex, highest-risk piece of the stack and shouldn't be built before the core idea is validated.

@modelcontextprotocol/sdk

The official TypeScript SDK for the Model Context Protocol — the same foundation almost every MCP server in the wild is built on, including Needle's. Used to expose the engine's resolution logic as callable tools (`get_object_provenance`, `search_by_seed`, `search_by_generator`) for AI coding agents like Claude Code or Cursor. Chosen specifically because it's the standard, well-documented SDK — using anything else would mean reinventing protocol-compliance work that's already solved.

tsup

Bundles the finished package for npm publishing. Chosen over a raw Vite library-mode build or Rollup config for its minimal configuration overhead — a handful of lines is typically enough for a package this size. It's also a common choice among comparable small developer-tool packages (Zustand and Zod both use it), which means good community reference material if something goes wrong.

Vitest

Used for automated tests, particularly around the raycast/resolve logic. Chosen over Jest because it reuses Vite's own transform pipeline instead of running a separate one, which keeps test runs fast — meaningful on a constrained 2-core machine. Raycast hit-testing can be tested with a headless three.js scene, since it's pure geometry math and needs no actual GPU rendering to verify.

Stage-by-Stage Tooling

The stack grows incrementally — each row below adds only what that specific stage needs on top of everything already introduced, chosen to minimize how often anything needs to be swapped out later.

Stage	New Tooling Introduced	Deliberately Not Needed Yet
0 — Scope freeze	None — planning only.	Everything technical.
1 — Prove the mechanism	Plain TypeScript + three.js Raycaster; manual <code>userData</code> tagging on one generator function.	Babel/AST, Vite plugin, React UI, MCP SDK.
2 — Provenance convention	Still just TypeScript — defining the shared <code>{file, function, line, args}</code> record shape.	Same as Stage 1.
3 — Core engine + overlay (MVP)	React, Zustand, OutlinePass, Node fs + a minimal Vite plugin for local read/write, Babel (minimal parse, just to locate one argument's character range) + magic-string (to patch it).	Babel's full auto-tagging transform, MCP SDK.
4 — Dogfooding	Nothing new — Stage 3's stack applied to the real Procedural World Generator.	—
5 — Package polish	tsup (bundling), Vitest (tests), ESLint, GitHub Actions (auto-run Vitest).	Babel's full auto-tagging transform, MCP SDK.
6 — MCP / AI mode	@modelcontextprotocol/sdk, zod (runtime validation of MCP tool inputs — untrusted input now enters via agent tool calls).	Babel's full auto-tagging transform (still optional at this point).
6.5 — Full auto-instrumentation (optional, post-launch)	Babel's traverse + types modules extended into a full build-time AST transform that auto-tags every generator call — replacing manual <code>userData</code> tagging entirely.	—
7 — Ship	Nothing new — publishing what already exists to npm and GitHub.	—

Why This Order, Specifically

- **Stage 1 is intentionally almost tool-free.** If the core raycast-to-provenance mechanism doesn't work reliably, that has to be discoverable in days with a throwaway script — not after weeks spent wiring up a Vite plugin, a React UI, and an MCP server that turn out to sit on a broken foundation.
- **The riskiest, most complex Babel work — the full auto-tagging transform — comes last, not first.** That full instrumentation is the single hardest technical piece in the whole stack, so it's deliberately held until Stage 6.5. A much smaller, narrower use of Babel (just enough parsing to locate one argument's position in a file) is needed earlier, at Stage 3, to support the MVP's basic edit-and-write-back feature — worth distinguishing from the larger transform so the plan doesn't overstate how long Babel stays out of the picture.
- **MCP is added only once there's something real to expose.** The MCP SDK is straightforward to wire up once the engine already resolves objects correctly — there's no reason to introduce it earlier than Stage 6, since it has nothing to connect to before then.
- **Every tool choice favors low setup cost and reuse of what's already familiar** (Vite, TS, R3F) over novelty — deliberately, given the hardware constraints and the goal of finishing rather than sprawling.

Also Worth Adding

A small set of additional tools worth including alongside the core stack — none change the architecture, each earns its place for a specific, narrow reason:

- **zod (Stage 6):** validates the structured inputs to MCP tools like `edit_parameter()` and `search_by_seed()` at runtime. This is the one boundary in the system where input arrives from outside the developer's own typed code — an AI agent's tool call — so TypeScript's compile-time types alone aren't enough to guard it.
- **ts-morph (future option, not for MVP):** a higher-level TypeScript API worth considering only if editing needs grow beyond single-literal patches — renaming parameters, inserting imports, moving code, or generating helper functions. `magic-string` remains the right choice for the initial "change one literal" workflow; `ts-morph` is a later upgrade path, not a current pick.
- **A lightweight logging abstraction:** a thin wrapper around console output rather than scattered `console.log` calls, so debug output can be toggled on/off cleanly. Small, but developer tools live and die by how easy they are to debug themselves.
- **ESLint (Stage 5):** standard project-quality linting, added during package polish alongside `tsup` and `Vitest`.
- **GitHub Actions (Stage 5):** runs `Vitest` automatically on push, so test coverage is enforced rather than relying on remembering to run it locally.

None of these change the core architecture described earlier — they're additions that earn their place the same way every other tool in this document had to, not new technology introduced for its own sake. Beyond this list, the working recommendation is to freeze the stack and only extend it if implementation surfaces a concrete, specific need.